

Admin Master React 移行計画

目次

1. 対象画面（masterディレクトリ）
 2. React/TypeScript 導入方法
 3. アクセスからAPI呼び出しまでの流れ
 4. BEコントローラーの対応
 5. spa.blade.php の中身
 6. UIライブラリ・CSSの構成
 7. FEのディレクトリ構成
 8. AdminLTE の廃止方針
 9. ローカル環境でのFE確認方法
 10. masterのタスク分け（優先順）
-

対象画面（masterディレクトリ）

コントローラー	画面数	備考
AvailableService（利用可能サービス）	4画面	index/create/detail/edit
MembershipFee（会費）	4画面	index/create/detail/edit
OperatingCompany（運営会社）	3画面	index/create/edit
Permission（権限）	1画面	index
SystemSetting（システム設定）	3画面	index/create/edit
Tax（税率）	4画面	index/create/detail/edit
Term（利用規約）	3画面	detail/edit/edit_confirm
AdminUser（管理者ユーザー）	4画面	index/create/detail/edit
Account（アカウント）	複数	auth含む

合計：約26画面

1. React/TypeScript 導入方法

現状の課題

- Laravel Mix（Webpack 4系）で `--openseal-legacy-provider` が必要な古い構成
- Vue 2.x が入っている

推奨方針：Vite + React 19 への移行

```
npm install vite laravel-vite-plugin @vitejs/plugin-react
npm install react@19 react-dom@19
npm install react-router-dom
npm install axios
npm install -D typescript @types/react @types/react-dom
```

`vite.config.ts` を新設し、`webpack.mix.js` と並行稼働させる。既存のBladeはMix継続、React新画面はViteで処理する二段構えが移行期間中の現実的な選択。

vite.config.ts

```
import { defineConfig } from 'vite'
import laravel from 'laravel-vite-plugin'
import react from '@vitejs/plugin-react'
import path from 'path'

export default defineConfig({
  plugins: [
    laravel({
      input: ['resources/src/admin/main.tsx'],
      refresh: true,
    }),
    react(),
  ],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, 'resources/src/admin'),
    },
  },
})
```

tsconfig.json

shadcn/ui が `@/` エイリアスを使うため設定が必要。

```
{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["resources/src/admin/*"]
    }
  },
}
```

```
"include": ["resources/src"]
}
```

2. アクセスからAPI呼び出しまでの流れ

現状

```
ブラウザ: GET /admin/user
↓
routes/admin.php → AdminUserController::index()
↓
AdminUserService::index() → view('admin/master/admin_users/index', [...])
↓
HTMLをまるごとレスポンス → ブラウザ表示
```

React化後

```
ブラウザ: GET /admin/user
↓
routes/admin.php → AdminSpaController::index()
↓
return view('admin/spa') ← ReactアプリのエントリーポイントとなるBladeを1つ返すだけ
↓
Reactが起動 → React Router が /admin/user を見る
↓
pages/master/admin_users/Index.tsx をレンダリング
↓
useEffect内で GET /api/admin/master/users を叩く
↓
routes/admin_api.php → AdminUserApiController::index()
↓
return response()->json([...])
↓
ReactがJSONを受け取ってテーブルを描画
```

ルーティングの2層構造

層	担当	ファイル
サーバーサイド	<code>/admin/*</code> → SPAシェルを返す	<code>routes/admin.php</code>
フロントエンド	URLを見てどのコンポーネントを表示するか	<code>App.tsx</code> (React Router)
API	<code>/api/admin/*</code> → JSONを返す	<code>routes/admin_api.php</code>

3. BEコントローラーの対応

JSON返却だけでOK

サービス層はそのまま流用し、コントローラーだけをAPI用に薄く作る。

```
// 既存ルート (Blade) → そのまま残す (移行期間中)
Route::get('/master/available_services',
[AvailableServiceController::class, 'index']);

// 新規APIルート (React用) → 追加
Route::prefix('api/admin')->middleware(['auth:admin'])->group(function () {
    Route::apiResource('master/available_services',
    AvailableServiceApiController::class);
});
```

AdminSpaController

/admin/* へのアクセスを受け取り、SPAシェルを返すだけのシンプルなコントローラー。

```
class AdminSpaController extends Controller
{
    public function index()
    {
        return view('admin/spa');
    }
}
```

移行期間中のルーティング

```
// routes/admin.php

// React化済み → SPAシェルへ
Route::prefix('master')->group(function () {
    Route::get('/tax', [AdminSpaController::class, 'index']);
    Route::get('/tax/{any}', [AdminSpaController::class, 'index'])->
    where('any', '.*');
});

// 未対応 → 既存コントローラーへ (そのまま残す)
Route::get('/master/available_service', [AvailableServiceController::class,
'index']);
```

最終形 (全画面React化完了後)

```
// /admin/* は全部ここに流す（ワイルドカード1行）
Route::get('/{any}', [AdminSpaController::class, 'index'])->where('any',
'.*');
```

4. spa.blade.php の中身

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="csrf-token" content="{ {{ csrf_token() }} }">
  <title>共済会管理システム</title>
  @vite(['resources/src/admin/main.tsx'])
</head>
<body>
  <div id="app"></div>
</body>
</html>
```

main.tsx

```
import '../css/admin/app.css'
import { createRoot } from 'react-dom/client'
import App from './App'

createRoot(document.getElementById('app')!).render(<App />)
```

既存の admin.blade.php との違い

	既存 admin.blade.php	新 spa.blade.php
中身	AdminLTEのサイドバー・ナビなど大量のHTML	<div id="app"> の1行のみ
サイドバー	Bladeで描画	ReactコンポーネントとしてReact側で描画
データ	Bladeにサーバーから渡す	APIで取得

5. UIライブラリ・CSSの構成

UIライブラリ：Tailwind CSS + shadcn/ui

shadcn/ui はTailwind CSSの上に成り立つコンポーネント集。セットで導入する。

- インラインCSSは書かない
- 自前でCSSファイルを作らない
- コンポーネントのスタイルはすべてshadcn/uiのコンポーネントで完結させる
- 動的な値（DBから取得した色コード等）が必要な場合のみ例外的にインラインを許容

shadcn/ui 導入手順

```
# 依存パッケージ
npm install clsx tailwind-merge class-variance-authority lucide-react

# shadcn/ui 初期化（対話形式）
npx shadcn@latest init
```

初期化時の選択：

```
Which style would you like to use? > Default
Which color would you like to use as base color? > Slate
Would you like to use CSS variables for colors? > yes
```

`components.json` が生成される。パスを `resources/src/admin` に合わせて調整する。

```
{
  "rsc": false,
  "tsx": true,
  "aliases": {
    "components": "@components",
    "utils": "@lib/utils"
  }
}
```

コンポーネントの追加

必要なコンポーネントをその都度追加する。

```
npx shadcn@latest add button
npx shadcn@latest add table
npx shadcn@latest add form
npx shadcn@latest add input
npx shadcn@latest add dialog
npx shadcn@latest add pagination
npx shadcn@latest add badge
npx shadcn@latest add alert
```

追加すると `resources/src/admin/components/ui/` 配下にソースが直接コピーされる（ライブラリに依存しない）。

グローバルCSS

`resources/src/admin/css/app.css` に配置。

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

使用イメージ

```
import { Button } from '@components/ui/button'
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow }
from '@components/ui/table'

const AdminUserIndex = () => (
  <div>
    <Button>新規登録</Button>
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead>名前</TableHead>
          <TableHead>メール</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        <TableRow>
          <TableCell>山田太郎</TableCell>
          <TableCell>yamada@example.com</TableCell>
        </TableRow>
      </TableBody>
    </Table>
  </div>
)
```

既存の `styles.css` との関係

移行完了まで `public/css/styles.css` はそのまま残す。全画面React化完了後に削除する。

6. FEのディレクトリ構成

React関連のファイルはすべて `resources/src` 配下にまとめる。既存の `resources/js` や `resources/sass` は一切触らない。

```

resources/
├── src/                                     # React関連はすべてここ
│   ├── admin/
│   │   ├── main.tsx                       # エントリーポイント
│   │   ├── App.tsx                       # React Router設定
│   │   ├── api/                          # API呼び出し層
│   │   │   ├── client.ts                # axiosインスタンス (CSRFトークン設定)
│   │   │   └── master/
│   │   │       ├── adminUser.ts
│   │   │       ├── availableService.ts
│   │   │       └── ...
│   │   ├── components/                  # 共通UIコンポーネント
│   │   │   ├── Table/
│   │   │   ├── Form/
│   │   │   ├── Pagination/
│   │   │   └── Layout/
│   │   ├── pages/                      # ページコンポーネント
│   │   │   ├── master/
│   │   │   │   ├── admin_users/
│   │   │   │   │   ├── Index.tsx
│   │   │   │   │   ├── Create.tsx
│   │   │   │   │   ├── Detail.tsx
│   │   │   │   │   └── Edit.tsx
│   │   │   │   ├── available_services/
│   │   │   │   ├── membership_fees/
│   │   │   │   ├── operating_companies/
│   │   │   │   ├── permissions/
│   │   │   │   ├── system_settings/
│   │   │   │   ├── taxes/
│   │   │   │   └── terms/
│   │   │   └── ...
│   │   ├── types/                      # TypeScript型定義
│   │   │   ├── models/
│   │   │   │   ├── AdminUser.ts
│   │   │   │   ├── AvailableService.ts
│   │   │   │   └── ...
│   │   │   └── components/
│   │   │       ├── ui/                  # shadcn/uiのコンポーネント (自動生成)
│   │   │       │   ├── button.tsx
│   │   │       │   ├── table.tsx
│   │   │       │   └── ...
│   │   │   └── css/
│   │   │       └── app.css              # Tailwind + グローバルCSS
│   │   └── ...                          # 将来他のガード (Employee等) も同階層に追
├── js/                                   # 既存 (触らない)
├── sass/                                 # 既存 (触らない)
└── views/                                # 既存 (触らない)

```

7. AdminLTE の廃止方針

AdminLTEが担っている要素と代替

AdminLTEの要素	React化後の代替
サイドバー	Reactコンポーネントで自作
トップナビバー	Reactコンポーネントで自作
ページレイアウト	Reactコンポーネントで自作
Bootstrap CSS	shadcn/ui + Tailwind
Font Awesome (アイコン)	lucide-react (shadcn/uiに含まれる)
jQuery依存のUI部品	shadcn/uiのコンポーネント

移行の進め方

フェーズ1：React化と並行してレイアウトを自作

インフラ整備タスクの中で、AdminLTEのレイアウトをReactコンポーネントとして再実装する。

```
resources/src/admin/
├── components/
│   └── layout/
│       ├── AppLayout.tsx    # 全体レイアウト（サイドバー+メインエリア）
│       ├── Sidebar.tsx     # サイドバー
│       ├── TopNav.tsx      # トップナビバー
│       └── PageHeader.tsx  # パンくず・ページタイトル
```

サイドバーのメニュー構成は現在の `config/adminlte.php` の `menu` 設定を参考に移植する。

```
// components/layout/Sidebar.tsx
const menuItems = [
  { label: 'ダッシュボード', path: '/admin/home', icon: LayoutDashboard },
  {
    label: 'マスタ管理',
    icon: Settings,
    children: [
      { label: '税率', path: '/admin/master/tax' },
      { label: '利用可能サービス', path: '/admin/master/available_services' }
    ]
  },
],
```

フェーズ2：React化済み画面はAdminLTEなしで動作

`spa.blade.php` はAdminLTEのCSSを読み込まないため、React化した画面は最初からAdminLTEフリーになる。移行期間中は以下が混在するが問題なく共存できる。

画面	レイアウト
React化済み	自作Reactレイアウト（AdminLTEなし）
未移行（Blade）	AdminLTEのまま

フェーズ3：全画面React化完了後にAdminLTE完全削除

```
# composerパッケージ削除
composer remove jeroennoten/laravel-adminlte

# npmパッケージ削除
npm uninstall admin-lte bootstrap jquery
```

合わせてAdminLTE関連のBladeレイアウトファイルも削除する。

8. ローカル環境でのFE確認方法

```
# ターミナル1: Laravel
php artisan serve

# ターミナル2: Vite (HMR対応)
npm run dev
```

ブラウザで <http://localhost:8000/admin/master/xxx> にアクセスして確認。

Viteの開発サーバー（:5173）は直接アクセスせず、Laravelサーバー経由でViteのアセットをロードする（[laravel-vite-plugin](#) がこれを担う）。

9. masterのタスク分け（優先順）

インフラ整備タスク（先行して実施）

1. Vite + React 19 + TypeScript 導入（webpack.mix.js と並行稼働）
2. shadcn/ui 導入（Tailwind CSS + shadcn/ui 初期化）
3. admin APIルートファイル作成 [routes/admin_api.php](#)
4. [AdminSpaController](#) 作成
5. [spa.blade.php](#) 作成
6. axiosクライアント設定（CSRFトークン、認証エラーハンドリング）
7. Reactレイアウトコンポーネント自作（AppLayout, Sidebar, TopNav, PageHeader）
8. 共通コンポーネント作成（shadcn/uiベース：Table, Pagination, Form, Alert）

各機能の移行タスク（インフラ完了後）

7. **Tax（税率）移行** — 画面がシンプル、最初の練習台に最適
8. **Permission（権限）移行** — 1画面のみ

9. **SystemSetting**（システム設定）移行
10. **OperatingCompany**（運営会社）移行
11. **MembershipFee**（会費）移行 — ロジックやや複雑
12. **AvailableService**（利用可能サービス）移行 — 最も複雑
13. **Term**（利用規約）移行 — 確認画面フローあり
14. **AdminUser・Account**移行 — 認証フロー含むため最後